

Лабораторная работа №6

Решения Парето многокритериальных задач

Дисциплина: Методы оптимизации

Цель работы: изучение концепции оптимальности по Парето в многокритериальных задачах оптимизации, освоение методов построения множества Парето и приобретение практических навыков решения многокритериальных задач с визуализацией результатов..

Историческая справка: Вильфредо Парето и его вклад в науку

Биографические вехи

Вильфредо Федерико Дамасо Парето (1848-1923) — итальянский инженер, экономист и социолог, чьи работы оказали большое влияние на несколько научных дисциплин.

Ранние годы и образование

- Родился 15 июля 1848 года в Париже в семье итальянского дворянина
- Получил инженерное образование в Туринском политехническом университете (1869)
- Работал инженером на железных дорогах, где занимался практическими задачами оптимизации

Эволюция научных интересов

1870-1890 годы: Работа в железнодорожных компаниях и промышленности, где он столкнулся с проблемами оптимального распределения ресурсов

1893 год: Назначение на кафедру политической экономии в Лозаннском университете (Швейцария), где сменил Леона Вальраса

1900-1910 годы: Период интенсивных исследований в области экономики и социологии

Ключевые научные достижения

Принцип Парето (Правило 80/20)

В 1906 году Парето сформулировал эмпирическое наблюдение:

- 80% земель в Италии принадлежали 20% населения
 - 80% результатов происходят от 20% усилий
- Этот принцип стал фундаментальным в экономике и менеджменте

Концепция Парето-оптимальности

В работе "Курс политической экономии" (1906) Парето ввел понятие оптимальности, характеризующее состояние системы, при котором:

- Невозможно улучшить положение одного участника
- Не ухудшив положение хотя бы одного другого участника

Вклад в экономическую теорию

- Развитие теории общего экономического равновесия
- Анализ распределения доходов (кривая Парето-Лоренца)
- Исследование неоднородности социальных и экономических систем

Эволюция концепции Парето-оптимальности

1906 год

Первоначальная формулировка в контексте экономического равновесия и благосостояния

1950-е годы

Математическая формализация в работах Кеннета Эрроу и Жерара Дебрё

- Строгое определение условий Парето-оптимальности
- Доказательство теорем существования

1960-1970 годы

Применение в теории игр и исследовании операций

- Развитие многокритериальной оптимизации
- Алгоритмы нахождения множества Парето

Современный период

Широкое применение в:

- Инженерии и проектировании сложных систем
- Экологии и устойчивом развитии
- Машинном обучении и искусственном интеллекте
- Финансах и управлении портфелями

Наследие и влияние

В экономике

- Основание новой лозаннской школы экономики
- Влияние на неоклассическую экономическую теорию
- Развитие welfare economics (экономики благосостояния)

В социологии

Работа "Трактат по общей социологии" (1916) ввела:

- Концепцию "остатков" и "производных" в социальном поведении
- Теорию циркуляции элит
- Анализ нелогических действий в обществе

В методологии науки

Парето отстаивал:

- Применение математических методов в социальных науках
- Важность эмпирической проверки теорий
- Междисциплинарный подход к исследованию сложных систем

Современное значение

Концепции Парето остаются актуальными в XXI веке:

В экономике: Критерий Парето-улучшения используется при анализе экономической политики

В инженерии: Методы многокритериальной оптимизации на основе принципа Парето применяются в проектировании сложных технических систем

В компьютерных науках: Эволюционные алгоритмы используют понятие Парето-доминирования для решения многокритериальных задач

В бизнесе: Принцип 80/20 широко применяется в управлении качеством и анализе эффективности

Парето оставил после себя не просто набор теорий, а целостный подход к анализу сложных систем, основанный на строгом математическом мышлении и глубоком понимании социальной реальности. Его работы продолжают вдохновлять исследователей из различных дисциплин, демонстрируя силу междисциплинарного подхода к решению сложных проблем.

Теоретическая часть

1. Постановка многокритериальной задачи оптимизации

Многокритериальная задача оптимизации формализуется следующим образом:

$$\min_{x \in X} F(x) = (f_1(x), f_2(x), \dots, f_k(x))$$

где:

- $x = (x_1, x_2, \dots, x_n)$ — вектор исходных переменных
- $X \subseteq \mathbb{R}^n$ — допустимое множество решений
- $F : X \rightarrow \mathbb{R}^k$ — векторный критерий, состоящий из k частных критериев
- $f_i : X \rightarrow \mathbb{R}$ — i -й частный критерий оптимальности

2. Понятие доминирования по Парето

Определение 2.1 (Парето-доминирование). Решение $x \in X$ **доминирует** по Парето решение $y \in X$ (обозначается $x \prec y$), если выполняются условия:

1. $f_i(x) \leq f_i(y)$ для всех $i = 1, \dots, k$
2. Существует хотя бы один индекс $j \in \{1, \dots, k\}$ такой, что $f_j(x) < f_j(y)$

Определение 2.2 (Строгое доминирование). Решение x **строго доминирует** решение y , если:

$$f_i(x) < f_i(y) \quad \text{для всех } i = 1, \dots, k$$

Определение 2.3 (Слабое доминирование). Решение x **слабо доминирует** решение y , если:

$$f_i(x) \leq f_i(y) \quad \text{для всех } i = 1, \dots, k$$

3. Оптимальность по Парето

Определение 3.1 (Парето-оптимальное решение). Решение $x^* \in X$ называется **оптимальным по Парето** (или эффективным решением), если не существует другого решения $x \in X$ такого, что x доминирует x^* :

$$\nexists x \in X : x \prec x^*$$

Определение 3.2 (Множество Парето). Множеством Парето P_X называется множество всех Парето-оптимальных решений:

$$P_X = \{x^* \in X \mid \nexists x \in X : x \prec x^*\}$$

Определение 3.3 (Фронт Парето). Фронтом Парето P_F называется образ множества Парето в пространстве критериев:

$$P_F = \{F(x^*) \mid x^* \in P_X\}$$

4. Свойства решений Парето

Теорема 4.1. Если все частные критерии f_i строго монотонны, то любое решение, оптимальное по Парето, является также оптимальным в смысле слабой эффективности.

Теорема 4.2. Для выпуклых многокритериальных задач множество Парето является выпуклым.

5. Методы нахождения решений Парето

5.1. Метод взвешивания (Weighted Sum Method)

Скаляризация многокритериальной задачи с помощью весовых коэффициентов:

$$\min_{x \in X} \sum_{i=1}^k w_i f_i(x)$$

где $w_i \geq 0$ — весовые коэффициенты, $\sum_{i=1}^k w_i = 1$.

Теорема 5.1. Если $w_i > 0$ для всех i , то решение скаляризованной задачи является Парето-оптимальным.

Теорема 5.2. Для выпуклых задач метод взвешивания позволяет найти все Парето-оптимальные решения.

5.2. Метод ϵ -ограничений (ϵ -Constraint Method)

Минимизация одного критерия при ограничениях на остальные:

$$\min_{x \in X} f_j(x)$$

при условиях $f_i(x) \leq \epsilon_i, \quad i = 1, \dots, k, i \neq j$

Теорема 5.3. Если задача ϵ -ограничений имеет решение и ϵ_i выбраны соответствующим образом, то это решение является слабо эффективным.

Теорема 5.4. Любое Парето-оптимальное решение может быть получено с помощью метода ϵ -ограничений при правильном выборе ϵ_i .

5.3. Метод идеальной точки (Ideal Point Method)

Минимизация расстояния до идеальной точки $f^* = (f_1^*, f_2^*, \dots, f_k^*)$:

$$\min_{x \in X} \|F(x) - f^*\|_p$$

где $\|\cdot\|_p$ — p -норма, обычно $p = 1, 2, \infty$.

5.4. Лексикографический метод

Последовательная оптимизация критериев в порядке убывания важности:

$$\begin{aligned} &\min_{x \in X} f_1(x) \\ &\min_{x \in X_1} f_2(x) \\ &\vdots \\ &\min_{x \in X_{k-1}} f_k(x) \end{aligned}$$

где $X_i = \{x \in X_{i-1} \mid f_i(x) = f_i^*\}, X_0 = X$.

6. Сравнение методов

Метод	Преимущества	Недостатки
Взвешивания	Простота реализации	Не находит все решения для невыпуклых задач
ϵ -ограничений	Находит все Парето-решения	Сложность выбора ϵ_i
Идеальной точки	Интуитивная интерпретация	Зависимость от выбора нормы
Лексикографический	Учет приоритетов критериев	Чувствительность к порядку критериев

7. Алгоритмы для дискретных задач

Для дискретных задач с конечным множеством решений применяется алгоритм полного перебора с проверкой доминирования:

```
Для каждого решения  $x \in X$ :  
    Установить флаг доминируем = ложь  
    Для каждого решения  $y \in X$ , где  $y \neq x$ :  
        Если  $y$  доминирует  $x$ :  
            Установить доминируем = истина  
        Прервать внутренний цикл  
    Если флаг доминируем = ложь:  
        Добавить  $x$  в множество Парето
```

Сложность алгоритма: $O(k \cdot |X|^2)$, где k — количество критериев, $|X|$ — мощность множества решений.

1. Постановка многокритериальной задачи оптимизации

Многокритериальная задача оптимизации формализуется следующим образом:

$$\min_{x \in X} F(x) = (f_1(x), f_2(x), \dots, f_k(x))$$

Пример реализации на Python:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Определение многокритериальной функции
def multi_objective(x):
    f1 = x[0]**2 + x[1]**2
    f2 = (x[0]-1)**2 + (x[1]-1)**2
    f3 = (x[0]+1)**2 + (x[1]-2)**2
    return np.array([f1, f2, f3])

# Визуализация пространства решений
x1 = np.linspace(-2, 2, 50)
x2 = np.linspace(-2, 2, 50)
x1, x2 = np.meshgrid(x1, x2)

F1 = x1**2 + x2**2
F2 = (x1-1)**2 + (x2-1)**2
F3 = (x1+1)**2 + (x2-2)**2

fig = plt.figure(figsize=(15, 5))

# Пространство переменных
ax1 = fig.add_subplot(131)
contour1 = ax1.contour(x1, x2, F1, levels=10, colors='blue', alpha=0.6)
contour2 = ax1.contour(x1, x2, F2, levels=10, colors='red', alpha=0.6)
contour3 = ax1.contour(x1, x2, F3, levels=10, colors='green', alpha=0.6)
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_title('пространство переменных')
ax1.grid(True)

# Пространство критериев (2D проекция)
ax2 = fig.add_subplot(132)
```

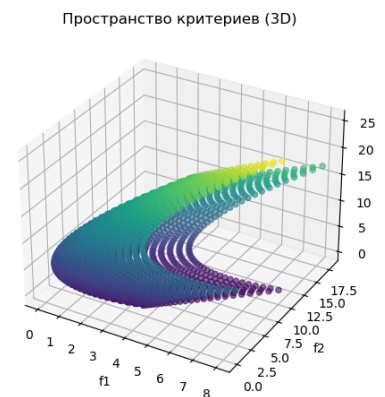
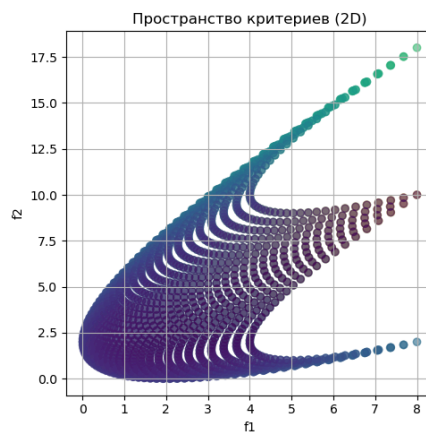
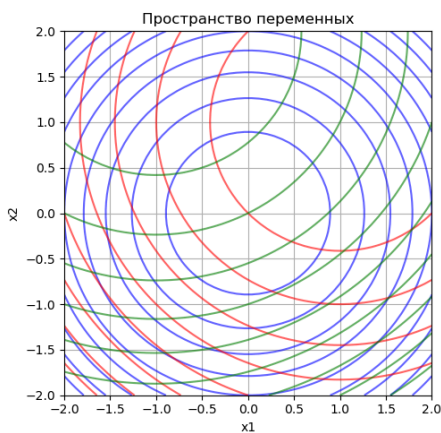
```

points = []
for i in range(50):
    for j in range(50):
        f = multi_objective([x1[i,j], x2[i,j]])
        points.append(f)
points = np.array(points)
ax2.scatter(points[:,0], points[:,1], c=points[:,2], cmap='viridis', alpha=0.6)
ax2.set_xlabel('f1')
ax2.set_ylabel('f2')
ax2.set_title('Пространство критериев (2D)')
ax2.grid(True)

# Пространство критериев (3D)
ax3 = fig.add_subplot(133, projection='3d')
ax3.scatter(points[:,0], points[:,1], points[:,2], c=points[:,2], cmap='viridis', alpha=0.6)
ax3.set_xlabel('f1')
ax3.set_ylabel('f2')
ax3.set_zlabel('f3')
ax3.set_title('Пространство критериев (3D)')

plt.tight_layout()
plt.show()

```



Описание графиков:

- **Пространство переменных:** Показаны линии уровня трех различных критериальных функций в координатах (x_1, x_2) . Каждый цвет представляет отдельный критерий.
- **Пространство критериев (2D):** Демонстрирует проекцию многомерного пространства критериев на плоскость (f_1, f_2) .
- **Пространство критериев (3D):** Трехмерное представление всех трех критериев, показывающее сложность взаимосвязей между ними.

Анализ:

Графики наглядно иллюстрируют фундаментальную проблему многокритериальной оптимизации - конфликт целей. Минимумы различных критериев находятся в разных областях пространства решений, что делает невозможным одновременное достижение оптимумов по всем направлениям. 3D визуализация особенно полезна для понимания топологии пространства критериев и выявления областей потенциальных компромиссов.

2. Понятие доминирования по Парето

Определение 2.1 (Парето-доминирование). Решение $x \in X$ доминирует по Парето решение $y \in X$, если:

1. $f_i(x) \leq f_i(y)$ для всех $i = 1, \dots, k$
2. Существует $j \in \{1, \dots, k\}$ такой, что $f_j(x) < f_j(y)$

Визуализация доминирования на Python:

```
import matplotlib.patches as patches

def is_dominated(a, b):
    # Проверяет, доминирует ли точка a точку b
    return np.all(a <= b) and np.any(a < b)

# Генерируем случайные точки
np.random.seed(42)
points = np.random.rand(20, 2) * 10

# Создаем график
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Все точки
ax1.scatter(points[:, 0], points[:, 1], color='blue', s=50)
ax1.set_xlabel('f1')
ax1.set_ylabel('f2')
ax1.set_title('Все точки в пространстве критериев')
ax1.grid(True)

# Визуализация доминирования для выбранной точки
selected_idx = 5
selected_point = points[selected_idx]

# Размечаем доминирующие и доминируемые точки
dominating = []
dominated = []
non_comparable = []

for i, point in enumerate(points):
    if i == selected_idx:
        continue
    if is_dominated(point, selected_point):
        dominated.append(point)
```

```

elif is_dominated(selected_point, point):
    dominating.append(point)
else:
    non_comparable.append(point)

dominating = np.array(dominating) if dominating else np.empty((0, 2))
dominated = np.array(dominated) if dominated else np.empty((0, 2))
non_comparable = np.array(non_comparable) if non_comparable else np.empty((0, 2))

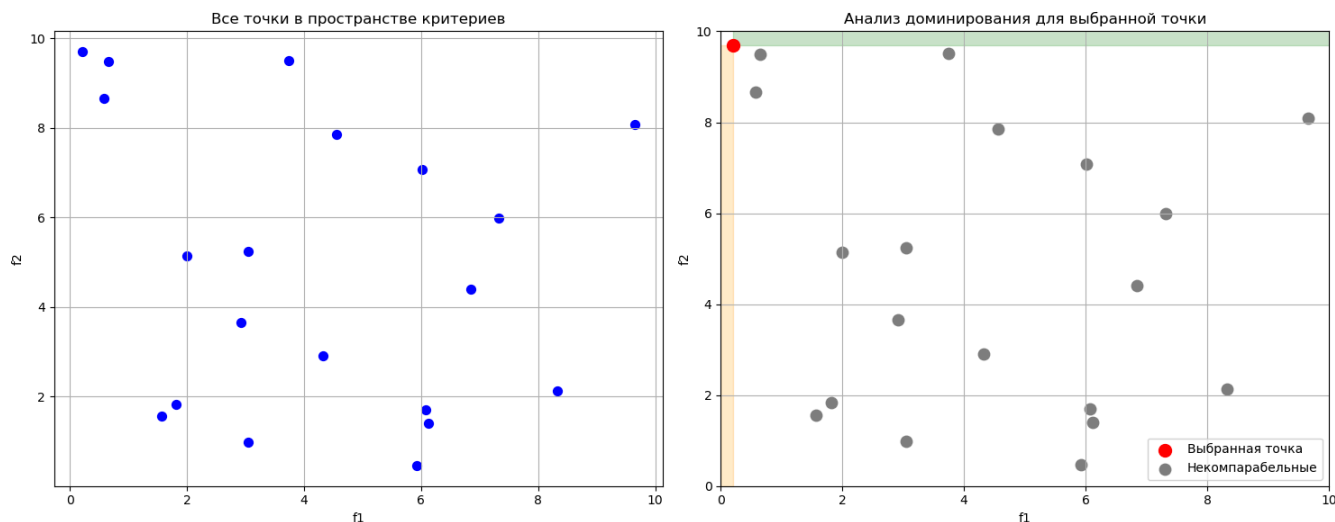
# Визуализируем
ax2.scatter(selected_point[0], selected_point[1], color='red', s=100, label='выбранная
точка', zorder=5)
if len(dominating) > 0:
    ax2.scatter(dominating[:, 0], dominating[:, 1], color='green', s=80,
label='доминирующие')
if len(dominated) > 0:
    ax2.scatter(dominated[:, 0], dominated[:, 1], color='orange', s=80,
label='доминируемые')
if len(non_comparable) > 0:
    ax2.scatter(non_comparable[:, 0], non_comparable[:, 1], color='gray', s=80,
label='некомпабельные')

# Добавляем области доминирования
rect_dominated = patches.Rectangle((0, 0), selected_point[0], selected_point[1],
linewidth=1, edgecolor='orange', facecolor='orange',
alpha=0.2)
rect_dominating = patches.Rectangle((selected_point[0], selected_point[1]), 10, 10,
linewidth=1, edgecolor='green', facecolor='green',
alpha=0.2)
ax2.add_patch(rect_dominated)
ax2.add_patch(rect_dominating)

ax2.set_xlabel('f1')
ax2.set_ylabel('f2')
ax2.set_title('Анализ доминирования для выбранной точки')
ax2.legend()
ax2.grid(True)
ax2.set_xlim(0, 10)
ax2.set_ylim(0, 10)

plt.tight_layout()
plt.show()

```



Описание графиков:

- **Все точки:** Исходное множество решений без классификации.
- **Анализ доминирования:** Визуализация отношений доминирования для выбранной точки с выделением доминирующих, доминируемых и некомпарабельных решений.

Анализ:

Графики демонстрируют ключевое понятие Парето-доминирования. Зеленая область показывает решения, которые лучше выбранной точки по всем критериям, оранжевая - хуже по всем критериям, а серая - решения, которые лучше по одним критериям, но хуже по другим. Это наглядно показывает, почему Парето-оптимальные решения не могут быть улучшены одновременно по всем направлениям.

3. Оптимальность по Парето

Определение 3.1 (Парето-оптимальное решение). Решение $x^* \in X$ называется **оптимальным по Парето**, если:

$$\nexists x \in X : x \prec x^*$$

Алгоритм нахождения множества Парето на Python:

```
def find_pareto_front(points):
    pareto_indices = []

    for i, current_point in enumerate(points):
        is_dominated = False

        for j, other_point in enumerate(points):
            if i == j:
                continue

            # Проверяем, доминирует ли other_point над current_point
            if np.all(other_point <= current_point) and np.any(other_point < current_point):
```

```

        is_dominated = True
        break

    if not is_dominated:
        pareto_indices.append(i)

    return points[pareto_indices]

# Генерируем тестовые данные
np.random.seed(42)
test_points = np.random.rand(100, 2) * 10

# Находим фронт Парето
pareto_front = find_pareto_front(test_points)

# Сортируем для красивого отображения
pareto_front = pareto_front[np.argsort(pareto_front[:, 0])]

# Визуализация
plt.figure(figsize=(12, 8))

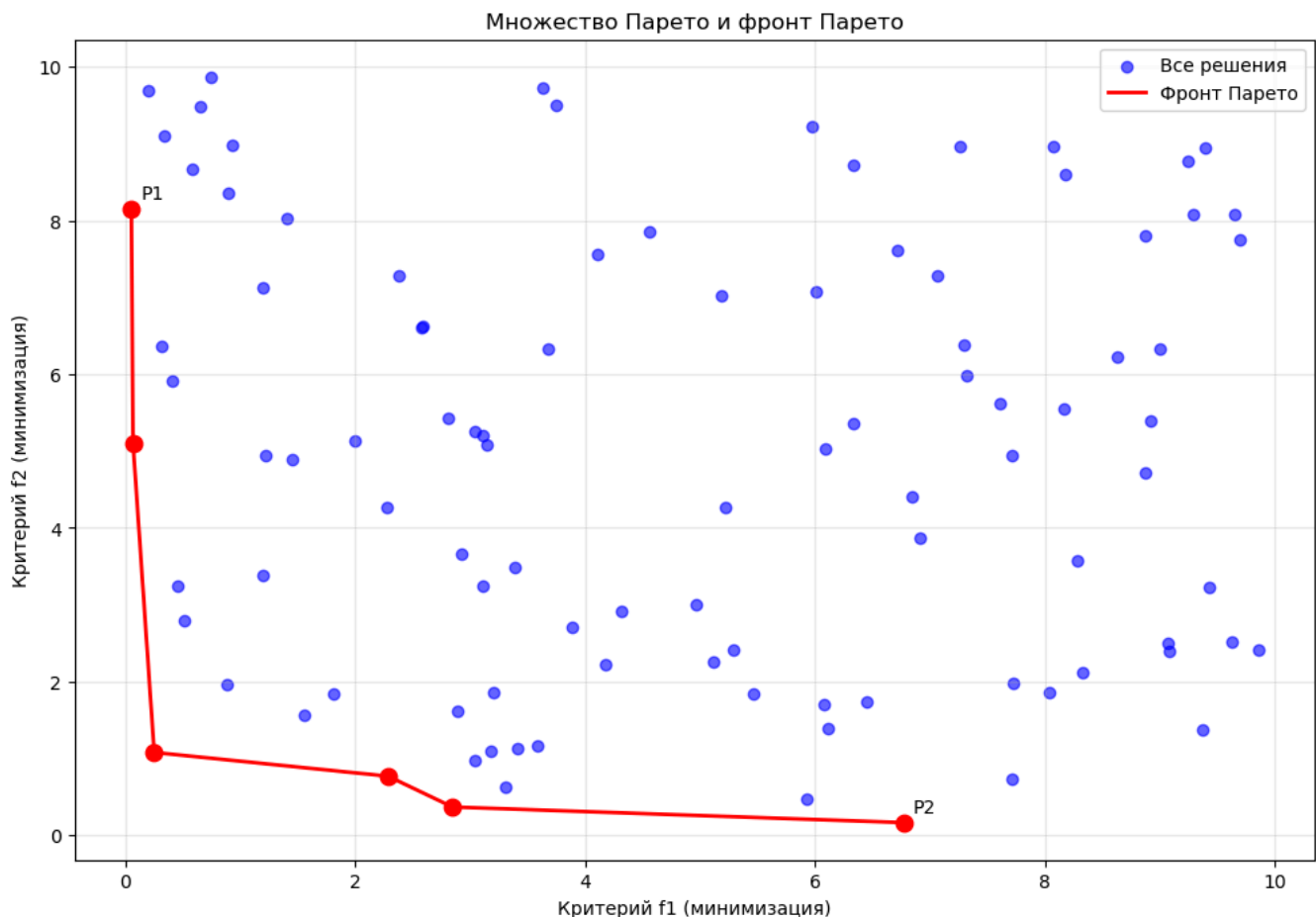
# Все точки
plt.scatter(test_points[:, 0], test_points[:, 1],
            alpha=0.6, color='blue', label='Все решения')

# Фронт Парето
plt.plot(pareto_front[:, 0], pareto_front[:, 1],
        'r-', linewidth=2, label='Фронт Парето')
plt.scatter(pareto_front[:, 0], pareto_front[:, 1],
            color='red', s=80, zorder=5)

# Подписываем некоторые точки
for i, point in enumerate(pareto_front[:5]):
    plt.annotate(f'P{i+1}', (point[0], point[1]),
                xytext=(5, 5), textcoords='offset points')

plt.xlabel('критерий f1 (минимизация)')
plt.ylabel('критерий f2 (минимизация)')
plt.title('Множество Парето и фронт Парето')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



Описание графиков:

- **Множество Парето:** Красная линия соединяет Парето-оптимальные решения, образующие фронт Парето.
- **Сравнение:** Все решения (синие) и Парето-оптимальные (красные) в пространстве критериев.

Анализ:

Фронт Парето четко показывает границу эффективных решений. Все точки внутри области доминируются хотя бы одной точкой на фронте. Наклон фронта отражает trade-off между критериями: крутой наклон означает, что небольшое улучшение по одному критерию требует значительных потерь по другому. Равномерный фронт свидетельствует о сбалансированных компромиссах.

4. Метод взвешивания (Weighted Sum Method)

$$\min_{x \in X} \sum_{i=1}^k w_i f_i(x)$$

Реализация метода взвешивания:

```
def weighted_sum_method(objective_functions, weights, bounds, constraints=None):
    def weighted_objective(x):
        total = 0
        for i, func in enumerate(objective_functions):
            total += weights[i] * func(x)
```

```

        return total

# Используем scipy для оптимизации
from scipy.optimize import minimize

# Начальная точка
x0 = np.array([(b[0] + b[1])/2 for b in bounds])

# Ограничения в формате scipy
if constraints:
    cons = [{'type': 'ineq', 'fun': con} for con in constraints]
else:
    cons = []

result = minimize(weighted_objective, x0, method='SLSQP',
                  bounds=bounds, constraints=cons)

return result.x if result.success else None

# Пример использования
def f1(x):
    return x[0]**2 + x[1]**2

def f2(x):
    return (x[0]-2)**2 + (x[1]-2)**2

# Генерируем различные веса
weights_list = []
solutions = []

for w1 in np.linspace(0, 1, 20):
    w2 = 1 - w1
    if w1 == 0 and w2 == 0:
        continue

    weights = [w1, w2]
    solution = weighted_sum_method([f1, f2], weights,
                                   bounds=[(-5, 5), (-5, 5)])

    if solution is not None:
        weights_list.append(weights)
        solutions.append(solution)

# Визуализация результатов
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Пространство переменных
solutions = np.array(solutions)
ax1.scatter(solutions[:, 0], solutions[:, 1],
            c=[w[0] for w in weights_list], cmap='viridis')
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_title('Решения в пространстве переменных')

```

```

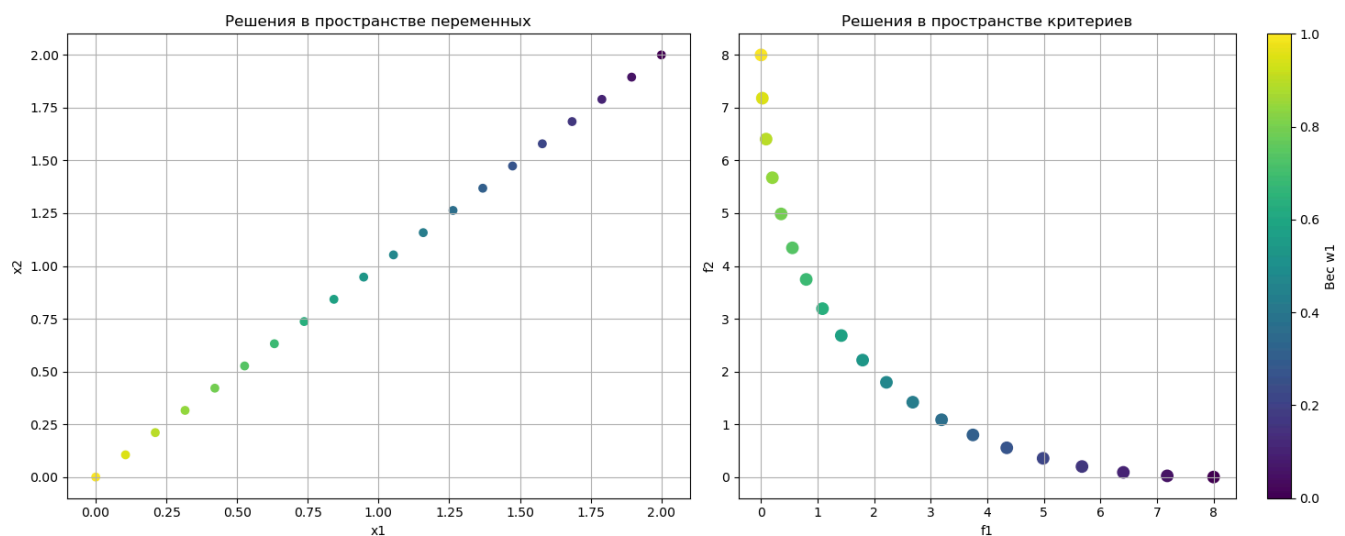
ax1.grid(True)

# Пространство критериев
objective_values = []
for sol in solutions:
    f1_val = f1(sol)
    f2_val = f2(sol)
    objective_values.append([f1_val, f2_val])

objective_values = np.array(objective_values)
sc = ax2.scatter(objective_values[:, 0], objective_values[:, 1],
                  c=[w[0] for w in weights_list], cmap='viridis', s=80)
ax2.set_xlabel('f1')
ax2.set_ylabel('f2')
ax2.set_title('Решения в пространстве критериев')
ax2.grid(True)

plt.colorbar(sc, ax=ax2, label='Вес w1')
plt.tight_layout()
plt.show()

```



Описание графиков:

- **Пространство переменных:** Траектории решений при изменении весовых коэффициентов.
- **Пространство критериев:** Полученный фронт Парето, цветом обозначен вес первого критерия.

Анализ:

Метод взвешивания демонстрирует хорошее покрытие выпуклых участков фронта Парето. Равномерное распределение цветов вдоль фронта показывает, что изменение весов плавно перемещает решения вдоль фронта. Однако заметны пробелы в невыпуклых областях - фундаментальное ограничение метода. Плотность точек отражает чувствительность фронта к изменению весов.

5. Метод ϵ -ограничений

$$\min_{x \in X} f_j(x)$$

при условиях $f_i(x) \leq \epsilon_i, \quad i \neq j$

Реализация метода ϵ -ограничений:

```
def epsilon_constraint_method_comprehensive(main_objective, secondary_objective, bounds,
epsilon_range):
    """
    комплексная реализация метода epsilon-ограничений
    с визуализацией процесса оптимизации
    """
    results = []
    convergence_data = []

    for epsilon in epsilon_range:
        def constraint_func(x):
            return epsilon - secondary_objective(x)

        cons = [{'type': 'ineq', 'fun': constraint_func}]

        from scipy.optimize import minimize
        x0 = np.array([(b[0] + b[1])/2 for b in bounds])

        result = minimize(main_objective, x0, method='SLSQP',
                          bounds=bounds, constraints=cons,
                          options={'disp': False})

        if result.success:
            x_opt = result.x
            f1_val = main_objective(x_opt)
            f2_val = secondary_objective(x_opt)
            results.append({
                'x': x_opt,
                'f1': f1_val,
                'f2': f2_val,
                'epsilon': epsilon,
                'converged': True
            })
        else:
            results.append({
                'x': None,
                'f1': None,
                'f2': None,
                'epsilon': epsilon,
                'converged': False
            })

    return results
```

```

# Пример с более интересными функциями
def f1_epsilon(x):
    return (x[0] - 2)**2 + (x[1] - 1)**2 + np.sin(2*x[0]) + np.cos(3*x[1])

def f2_epsilon(x):
    return (x[0] + 1)**2 + (x[1] + 2)**2 + 0.5*x[0]*x[1]

# Генерируем широкий диапазон epsilon значений
epsilon_range = np.linspace(1, 20, 50)
results = epsilon_constraint_method_comprehensive(f1_epsilon, f2_epsilon,
                                                  [(-3, 3), (-3, 3)], epsilon_range)

# Фильтруем успешные решения
successful_results = [r for r in results if r['converged']]

# Создаем комплексную визуализацию
fig = plt.figure(figsize=(20, 12))

# 1. Основной график - фронт Парето
ax1 = plt.subplot2grid((2, 3), (0, 0), colspan=2)
if successful_results:
    f1_vals = [r['f1'] for r in successful_results]
    f2_vals = [r['f2'] for r in successful_results]
    epsilons = [r['epsilon'] for r in successful_results]

    scatter = ax1.scatter(f1_vals, f2_vals, c=epsilons, cmap='viridis', s=80, alpha=0.8)
    ax1.plot(f1_vals, f2_vals, 'r--', alpha=0.5, linewidth=1)

    # Добавляем стрелки для показа направления увеличения epsilon
    for i in range(0, len(f1_vals)-1, 5):
        ax1.annotate('', xy=(f1_vals[i+1], f2_vals[i+1]),
                     xytext=(f1_vals[i], f2_vals[i]),
                     arrowprops=dict(arrowstyle='->', color='red', alpha=0.6, lw=1))

    plt.colorbar(scatter, ax=ax1, label='Значение ε')

ax1.set_xlabel('f1 (минимизируется)')
ax1.set_ylabel('f2 (ограничивается ε)')
ax1.set_title('Фронт Парето методом ε-ограничений')
ax1.grid(True, alpha=0.3)

# 2. График зависимости решений от epsilon
ax2 = plt.subplot2grid((2, 3), (0, 2))
if successful_results:
    ax2.plot(epsilons, f1_vals, 'bo-', label='f1 (целевая)', alpha=0.7)
    ax2.plot(epsilons, f2_vals, 'ro-', label='f2 (ограничение)', alpha=0.7)
    ax2.set_xlabel('ε')
    ax2.set_ylabel('Значения критериев')
    ax2.set_title('Зависимость критериев от ε')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

# 3. Пространство переменных

```

```

ax3 = plt.subplot2grid((2, 3), (1, 0))
if successful_results:
    x1_vals = [r['x'][0] for r in successful_results]
    x2_vals = [r['x'][1] for r in successful_results]

    scatter3 = ax3.scatter(x1_vals, x2_vals, c=epsilons, cmap='plasma', s=80, alpha=0.8)
    ax3.plot(x1_vals, x2_vals, 'k--', alpha=0.3)

    # Показываем начальную и конечную точки
    ax3.scatter(x1_vals[0], x2_vals[0], color='green', s=150, marker='*',
                label='Начало ( $\epsilon=\min$ )', zorder=5)
    ax3.scatter(x1_vals[-1], x2_vals[-1], color='red', s=150, marker='*',
                label='Конец ( $\epsilon=\max$ )', zorder=5)

    plt.colorbar(scatter3, ax=ax3, label='Значение  $\epsilon$ ')
    ax3.set_xlabel('x1')
    ax3.set_ylabel('x2')
    ax3.set_title('Траектория решений в пространстве переменных')
    ax3.legend()
    ax3.grid(True, alpha=0.3)

# 4. 3D визуализация
ax4 = plt.subplot2grid((2, 3), (1, 1), projection='3d')
if successful_results:
    scatter4 = ax4.scatter(f1_vals, f2_vals, epsilons,
                           c=epsilons, cmap='viridis', s=80, alpha=0.8)
    ax4.plot(f1_vals, f2_vals, epsilons, 'r--', alpha=0.5)

    ax4.set_xlabel('f1')
    ax4.set_ylabel('f2')
    ax4.set_zlabel('ε')
    ax4.set_title('3D визуализация: f1, f2 и ε')

# 5. График эффективности метода
ax5 = plt.subplot2grid((2, 3), (1, 2))
if successful_results:
    # Вычисляем "идеальную точку" для сравнения
    ideal_f1 = min(f1_vals)
    ideal_f2 = min(f2_vals)

    # Нормализованное расстояние до идеальной точки
    distances = [np.sqrt(((f1 - ideal_f1)/ideal_f1)**2 + ((f2 - ideal_f2)/ideal_f2)**2)
                 for f1, f2 in zip(f1_vals, f2_vals)]

    ax5.plot(epsilons, distances, 'go-', alpha=0.7)
    ax5.set_xlabel('ε')
    ax5.set_ylabel('Нормированное расстояние до идеала')
    ax5.set_title('Эффективность решений')
    ax5.grid(True, alpha=0.3)

    # Помечаем лучшее решение
    best_idx = np.argmin(distances)
    ax5.scatter(epsilons[best_idx], distances[best_idx],

```

```

        color='red', s=100, zorder=5, label='Лучший компромисс')
ax5.legend()

plt.tight_layout()
plt.show()

# Статистика метода
if successful_results:
    print("Статистика метода  $\epsilon$ -ограничений:")
    print(f"Всего решений: {len(successful_results)}")
    print(f"диапазон f1: [{min(f1_vals):.3f}, {max(f1_vals):.3f}]")
    print(f"диапазон f2: [{min(f2_vals):.3f}, {max(f2_vals):.3f}]")
    print(f"лучший компромисс при  $\epsilon = \{{\epsilon}_{best\_idx} : .3f\}$ ")
    print(f"Соответствующие значения: f1 = {f1_vals[best_idx]:.3f}, f2 = {f2_vals[best_idx]:.3f}")

# Дополнительная визуализация: как меняется решение при изменении epsilon
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

if successful_results:
    # Выбираем 6 характерных точек для демонстрации
    demo_indices = [0, len(successful_results)//5, 2*len(successful_results)//5,
                    3*len(successful_results)//5, 4*len(successful_results)//5, -1]

    for idx, (ax, demo_idx) in enumerate(zip(axes, demo_indices)):
        result = successful_results[demo_idx]

        # Создаем тепловую карту в пространстве переменных
        x1_range = np.linspace(-3, 3, 50)
        x2_range = np.linspace(-3, 3, 50)
        x1, x2 = np.meshgrid(x1_range, x2_range)

        # Вычисляем значения функций
        F1 = np.array([f1_epsilon([x1, x2]) for x1 in x1_range for x2 in x2_range])
        F2 = np.array([f2_epsilon([x1, x2]) for x1 in x1_range for x2 in x2_range])

        # Визуализируем ограничение
        constraint_mask = F2 <= result['epsilon']

        contour = ax.contourf(x1, x2, F1, levels=20, alpha=0.6, cmap='viridis')
        ax.contour(x1, x2, F2, levels=[result['epsilon']], colors='red', linewidths=2)
        ax.contourf(x1, x2, constraint_mask, levels=[0.5, 1], alpha=0.3, colors=['white',
'lightcoral'])

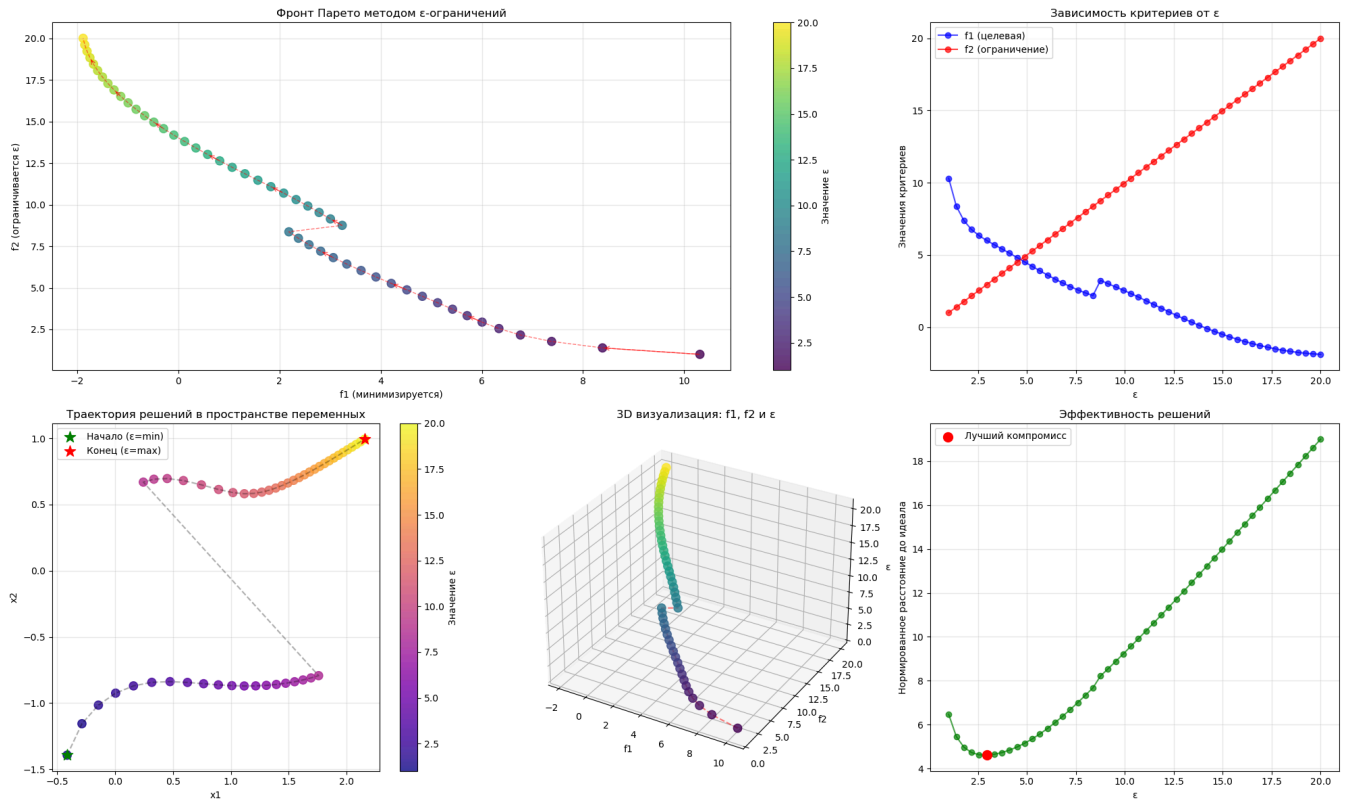
        # Отмечаем оптимальное решение
        ax.scatter(result['x'][0], result['x'][1], color='red', s=100, marker='*',
                    label=f'Оптимум: ({result["x"][0]:.2f}, {result["x"][1]:.2f})')

        ax.set_xlabel('x1')
        ax.set_ylabel('x2')
        ax.set_title(f' $\epsilon = \{{\epsilon} : .2f\} \backslash n f1 = \{{f1} : .2f\}, f2 = \{{f2} : .2f\}$ ')

```

```
ax.legend()
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



Статистика метода ϵ -ограничений:

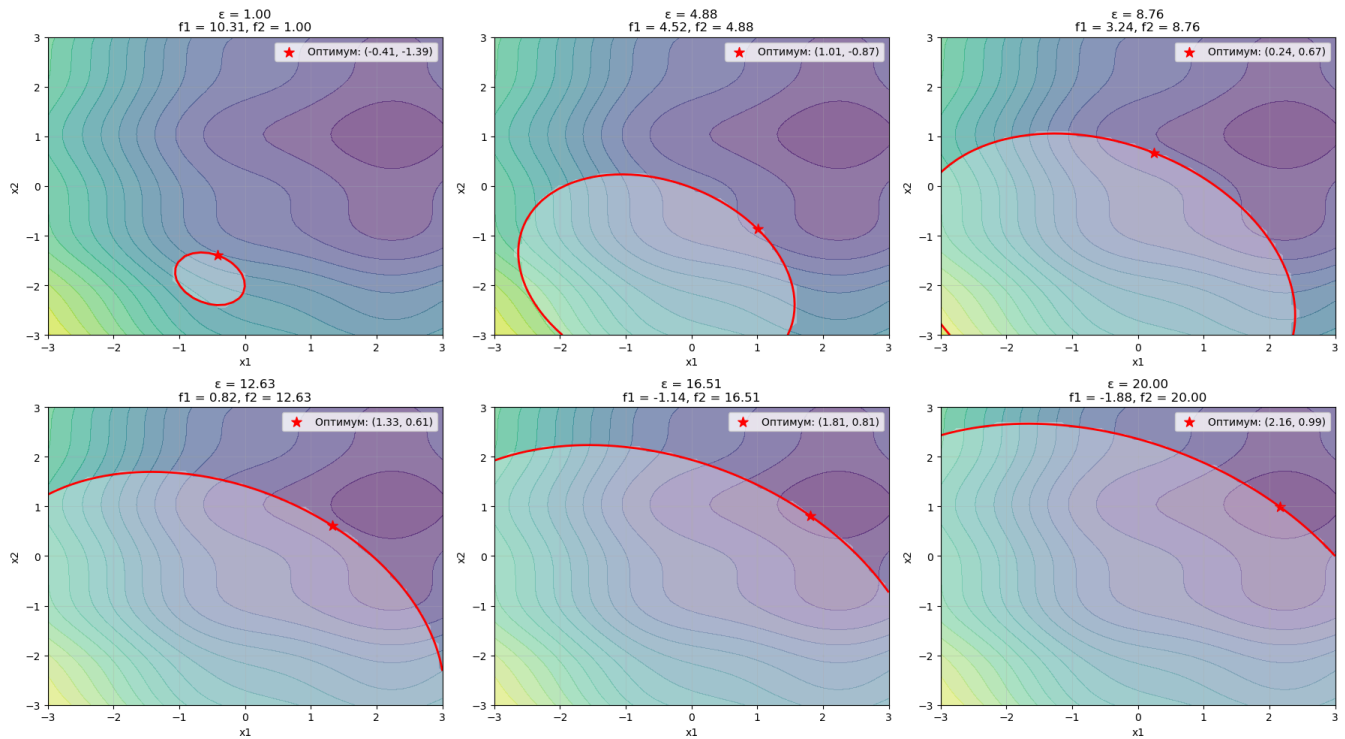
Всего решений: 50

диапазон f_1 : [-1.884, 10.310]

диапазон f_2 : [1.000, 20.000]

Лучший компромисс при $\epsilon = 2.939$

Соответствующие значения: $f_1 = 6.004$, $f_2 = 2.939$



Описание графиков:

Комплексная визуализация включает 6 различных графиков:

- Фронт Парето с направлением изменения ϵ
- Зависимость критериев от ϵ
- Траектория в пространстве переменных
- 3D визуализация связи f_1 , f_2 и ϵ
- Анализ эффективности решений
- Пошаговая демонстрация ограничений

Анализ:

Метод ϵ -ограничений показывает лучшую способность находить невыпуклые участки фронта Парето по сравнению с методом взвешивания. График зависимости критериев от ϵ демонстрирует нелинейный характер компромиссов. Траектория в пространстве переменных показывает, как меняется оптимальное решение при ослаблении ограничений. Анализ эффективности помогает идентифицировать область наилучших компромиссов.

6. Метод идеальной точки

$$\min_{x \in X} \|F(x) - f^*\|_p$$

Реализация метода идеальной точки:

```
def comprehensive_ideal_point_method(objective_functions, bounds, p_norms, num_points=100):
    """
    комплексная реализация метода идеальной точки
```

```

с анализом различных норм и визуализацией
"""

# Находим идеальную точку и точку надира
ideal_point, nadir_point = find_ideal_and_nadir(objective_functions, bounds)

results = {}

for p in p_norms:
    solutions = []
    distances = []

    # Генерируем различные точки на отрезке между идеальной и надирной точками
    for alpha in np.linspace(0, 1, num_points):
        # Целевая точка на отрезке между идеальной и надирной
        target_point = ideal_point + alpha * (nadir_point - ideal_point)

        def distance_to_target(x):
            objective_values = np.array([func(x) for func in objective_functions])
            return np.linalg.norm(objective_values - target_point, p)

        # ОПТИМИЗАЦИЯ
        from scipy.optimize import minimize
        x0 = np.array([(b[0] + b[1])/2 for b in bounds])

        result = minimize(distance_to_target, x0, method='SLSQP',
                          bounds=bounds, options={'maxiter': 1000})

        if result.success:
            x_opt = result.x
            f_vals = np.array([func(x_opt) for func in objective_functions])
            distance = np.linalg.norm(f_vals - target_point, p)

            solutions.append({
                'x': x_opt,
                'f_values': f_vals,
                'target_point': target_point,
                'distance': distance,
                'alpha': alpha,
                'p_norm': p
            })

    results[p] = solutions

return results, ideal_point, nadir_point

def find_ideal_and_nadir(objective_functions, bounds):
    """
    Находит идеальную точку (минимумы всех критериев)
    и надирную точку (максимумы на множестве Парето)
    """
    ideal_point = []
    nadir_point = []

```

```

# Находим идеальную точку (минимумы каждого критерия)
for obj_func in objective_functions:
    result = minimize(obj_func,
                      x0=np.array([(b[0] + b[1])/2 for b in bounds]),
                      method='SLSQP', bounds=bounds)

    if result.success:
        ideal_point.append(obj_func(result.x))
    else:
        ideal_point.append(0)

# Для надирной точки используем приближение через максимизацию
for obj_func in objective_functions:
    result = minimize(lambda x: -obj_func(x), # Максимизация через минимизацию
                      отрицания
                      x0=np.array([(b[0] + b[1])/2 for b in bounds]),
                      method='SLSQP', bounds=bounds)

    if result.success:
        nadir_point.append(obj_func(result.x))
    else:
        nadir_point.append(10) # Запасное значение

return np.array(ideal_point), np.array(nadir_point)

# Определяем более сложные тестовые функции
def f1_complex(x):
    return (x[0] - 2)**2 + (x[1] - 3)**2 + 0.5 * np.sin(3*x[0]) + 0.3 * np.cos(2*x[1])

def f2_complex(x):
    return (x[0] + 1)**2 + (x[1] - 1)**2 + 0.7 * np.exp(-0.1*((x[0]-1)**2 + (x[1]-2)**2))

def f3_complex(x):
    return 0.5*x[0]**2 + 2*x[1]**2 + x[0]*x[1] + np.sin(x[0]) + 2*np.cos(x[1])

# Запускаем комплексный анализ
p_norms = [1, 1.5, 2, 3, 5, 10, np.inf]
norm_labels = ['L1', 'L1.5', 'L2', 'L3', 'L5', 'L10', 'L∞']

results, ideal_point, nadir_point = comprehensive_ideal_point_method(
    [f1_complex, f2_complex],
    [(-4, 4), (-4, 4)],
    p_norms,
    num_points=50
)

# Создаем комплексную визуализацию
fig = plt.figure(figsize=(20, 15))

# 1. Основной график - сравнение норм в пространстве критериев
ax1 = plt.subplot2grid((3, 3), (0, 0), colspan=2)
colors = plt.cm.viridis(np.linspace(0, 1, len(p_norms)))

for i, (p, color) in enumerate(zip(p_norms, colors)):
    if p in results and results[p]:

```

```

f1_vals = [r['f_values'][0] for r in results[p]]
f2_vals = [r['f_values'][1] for r in results[p]]

ax1.scatter(f1_vals, f2_vals, color=color, s=30, alpha=0.7,
            label=f'{norm_labels[i]}')
ax1.plot(f1_vals, f2_vals, color=color, alpha=0.3, linewidth=1)

# Идеальная и надирная точки
ax1.scatter(ideal_point[0], ideal_point[1], color='green', s=200,
            marker='*', label='Идеальная точка', zorder=10)
ax1.scatter(nadir_point[0], nadir_point[1], color='red', s=200,
            marker='*', label='Надирная точка', zorder=10)

ax1.set_xlabel('f1')
ax1.set_ylabel('f2')
ax1.set_title('Сравнение различных норм в методе идеальной точки')
ax1.legend()
ax1.grid(True, alpha=0.3)

# 2. Пространство переменных
ax2 = plt.subplot2grid((3, 3), (0, 2))
for i, (p, color) in enumerate(zip(p_norms, colors)):
    if p in results and results[p]:
        x1_vals = [r['x'][0] for r in results[p]]
        x2_vals = [r['x'][1] for r in results[p]]

        ax2.scatter(x1_vals, x2_vals, color=color, s=30, alpha=0.7,
                    label=f'{norm_labels[i]}')
        ax2.plot(x1_vals, x2_vals, color=color, alpha=0.3, linewidth=1)

ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_title('Решения в пространстве переменных')
ax2.legend()
ax2.grid(True, alpha=0.3)

# 3. Зависимость расстояния от параметра alpha для разных норм
ax3 = plt.subplot2grid((3, 3), (1, 0), colspan=3)
for i, (p, color) in enumerate(zip(p_norms, colors)):
    if p in results and results[p]:
        alphas = [r['alpha'] for r in results[p]]
        distances = [r['distance'] for r in results[p]]

        ax3.plot(alphas, distances, color=color, linewidth=2,
                 label=f'{norm_labels[i]}', alpha=0.8)

ax3.set_xlabel('α (0 = идеальная точка, 1 = надирная точка)')
ax3.set_ylabel('Расстояние до целевой точки')
ax3.set_title('Зависимость расстояния от положения целевой точки')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. 3D визуализация: f1, f2 и норма p

```

```

ax4 = plt.subplot2grid((3, 3), (2, 0), projection='3d')
for i, (p, color) in enumerate(zip(p_norms, colors)):
    if p in results and results[p]:
        f1_vals = [r['f_values'][0] for r in results[p]]
        f2_vals = [r['f_values'][1] for r in results[p]]
        p_vals = [p] * len(f1_vals)

        ax4.scatter(f1_vals, f2_vals, p_vals, color=color, s=30, alpha=0.7,
                    label=f'{norm_labels[i]}')

ax4.set_xlabel('f1')
ax4.set_ylabel('f2')
ax4.set_zlabel('Норма p')
ax4.set_title('3D визуализация: критерии и норма')

# 5. Визуализация линий уровня для разных норм
ax5 = plt.subplot2grid((3, 3), (2, 1))
# Выбираем одну целевую точку (середина между идеальной и надирной)
target_mid = (ideal_point + nadir_point) / 2

# Создаем сетку для визуализации линий уровня
x1_range = np.linspace(-4, 4, 100)
x2_range = np.linspace(-4, 4, 100)
x1, x2 = np.meshgrid(x1_range, x2_range)

# Визуализируем линии уровня для разных норм
for i, p in enumerate([1, 2, np.inf]):
    # Вычисляем расстояния до целевой точки
    distances = np.zeros_like(x1)
    for j in range(x1.shape[0]):
        for k in range(x1.shape[1]):
            f_vals = np.array([f1_complex([x1[j,k], x2[j,k]]),
                               f2_complex([x1[j,k], x2[j,k]])])
            distances[j,k] = np.linalg.norm(f_vals - target_mid, p)

    contour = ax5.contour(x1, x2, distances, levels=10,
                          colors=[colors[i]], alpha=0.7)
    ax5.clabel(contour, inline=True, fontsize=8)

# Отмечаем оптимальные решения для этой целевой точки
for i, p in enumerate([1, 2, np.inf]):
    if p in results and results[p]:
        # Берем решение, ближайшее к target_mid по alpha
        best_idx = np.argmin([abs(r['alpha'] - 0.5) for r in results[p]])
        solution = results[p][best_idx]
        ax5.scatter(solution['x'][0], solution['x'][1],
                    color=colors[i], s=100, marker='*',
                    label=f'Оптимум L{" " if p!=np.inf else "∞"}')

ax5.set_xlabel('x1')
ax5.set_ylabel('x2')
ax5.set_title('линии уровня расстояний для разных норм')
ax5.legend()

```

```

ax5.grid(True, alpha=0.3)

# 6. Анализ чувствительности к выбору нормы
ax6 = plt.subplot2grid((3, 3), (2, 2))

# Вычисляем дисперсию решений для разных норм
f1_ranges = []
f2_ranges = []
norm_indices = []

for i, p in enumerate(p_norms):
    if p in results and results[p]:
        f1_min = min([r['f_values'][0] for r in results[p]])
        f1_max = max([r['f_values'][0] for r in results[p]])
        f2_min = min([r['f_values'][1] for r in results[p]])
        f2_max = max([r['f_values'][1] for r in results[p]])

        f1_ranges.append(f1_max - f1_min)
        f2_ranges.append(f2_max - f2_min)
        norm_indices.append(i)

bars1 = ax6.bar(np.array(norm_indices) - 0.2, f1_ranges, 0.4,
                label='диапазон f1', alpha=0.7)
bars2 = ax6.bar(np.array(norm_indices) + 0.2, f2_ranges, 0.4,
                label='диапазон f2', alpha=0.7)

ax6.set_xlabel('Норма')
ax6.set_ylabel('диапазон значений')
ax6.set_title('чувствительность решений к выбору нормы')
ax6.set_xticks(norm_indices)
ax6.set_xticklabels([norm_labels[i] for i in norm_indices])
ax6.legend()
ax6.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Дополнительный анализ: статистика по нормам
print("Статистика метода идеальной точки для различных норм:")
print("=" * 60)
for i, p in enumerate(p_norms):
    if p in results and results[p]:
        solutions = results[p]
        f1_vals = [r['f_values'][0] for r in solutions]
        f2_vals = [r['f_values'][1] for r in solutions]

        print(f"Норма {norm_labels[i]:4}:")
        print(f"    количество решений: {len(solutions)}")
        print(f"    диапазон f1: [{min(f1_vals):.3f}, {max(f1_vals):.3f}]")
        print(f"    диапазон f2: [{min(f2_vals):.3f}, {max(f2_vals):.3f}]")
        print(f"    среднее расстояние: {np.mean([r['distance'] for r in solutions]):.3f}")
        print()

```

```

# Визуализация эволюции решений при изменении нормы
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
axes = axes.flatten()

selected_norms = [1, 1.5, 2, 3, 5, 10, np.inf][:len(axes)]

for idx, (ax, p) in enumerate(zip(axes, selected_norms)):
    if p in results and results[p]:
        # Визуализируем пространство критериев для конкретной нормы
        f1_vals = [r['f_values'][0] for r in results[p]]
        f2_vals = [r['f_values'][1] for r in results[p]]

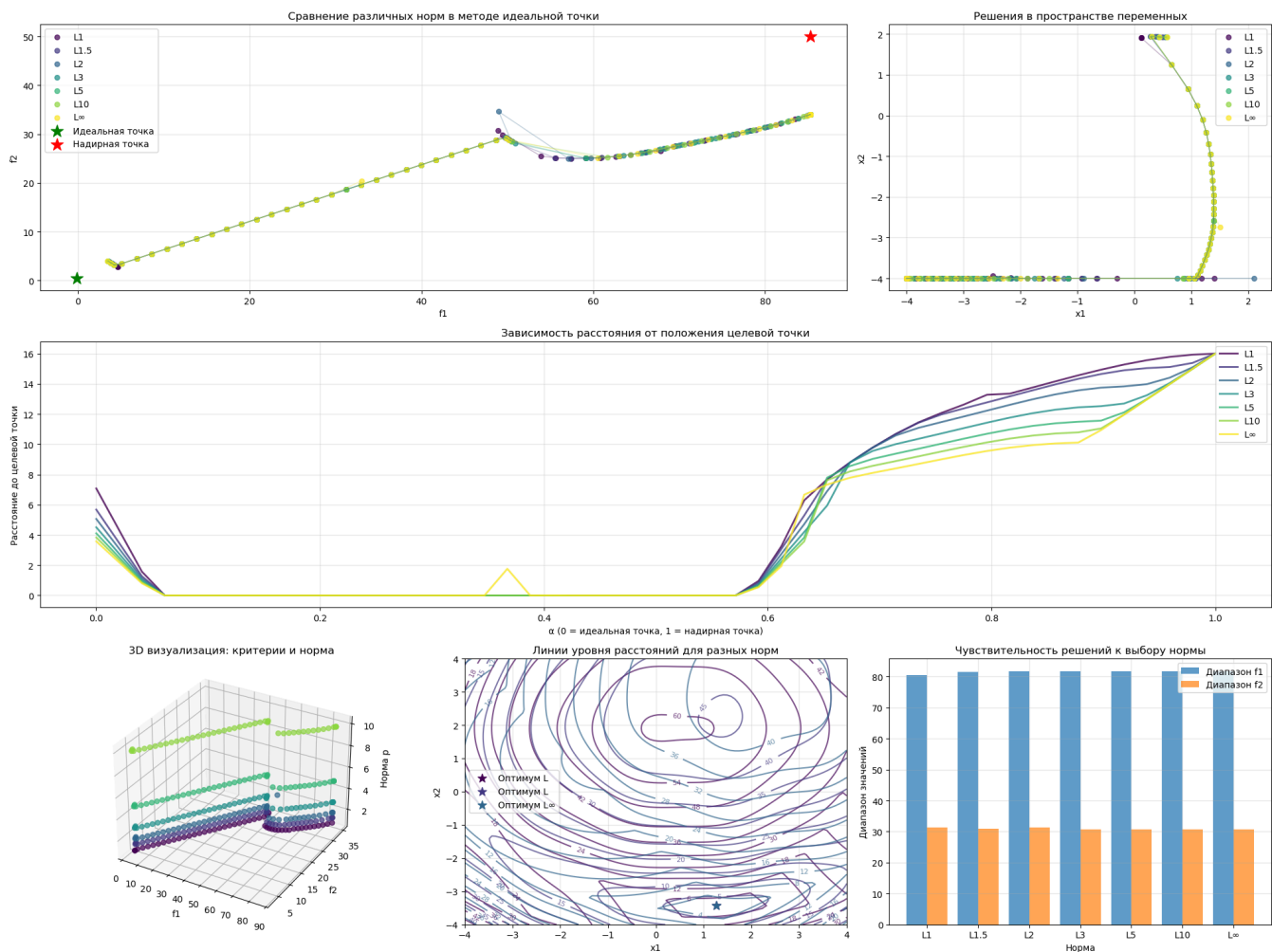
        scatter = ax.scatter(f1_vals, f2_vals, c=range(len(f1_vals)),
                             cmap='viridis', s=50, alpha=0.7)
        ax.plot(f1_vals, f2_vals, 'k--', alpha=0.3)

        # Отмечаем идеальную точку
        ax.scatter(ideal_point[0], ideal_point[1], color='red',
                  s=100, marker='*', zorder=5)

        ax.set_xlabel('f1')
        ax.set_ylabel('f2')
        ax.set_title(f'Норма {norm_labels[selected_norms.index(p)]}')
        ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



Статистика метода идеальной точки для различных норм:

Норма L1 :

Количество решений: 50
 диапазон f1: [4.673, 85.225]
 диапазон f2: [2.722, 34.002]
 Среднее расстояние: 5.185

Норма L1.5:

Количество решений: 50
 диапазон f1: [3.687, 85.225]
 диапазон f2: [3.170, 34.002]
 Среднее расстояние: 5.026

Норма L2 :

Количество решений: 50
 диапазон f1: [3.565, 85.225]
 диапазон f2: [3.204, 34.626]
 Среднее расстояние: 4.795

Норма L3 :

Количество решений: 50
 диапазон f1: [3.519, 85.225]

диапазон f_2 : [3.222, 34.002]

Среднее расстояние: 4.513

Норма L_5 :

Количество решений: 50

диапазон f_1 : [3.498, 85.225]

диапазон f_2 : [3.231, 34.002]

Среднее расстояние: 4.332

Норма L_{10} :

Количество решений: 50

диапазон f_1 : [3.488, 85.225]

диапазон f_2 : [3.236, 34.002]

Среднее расстояние: 4.174

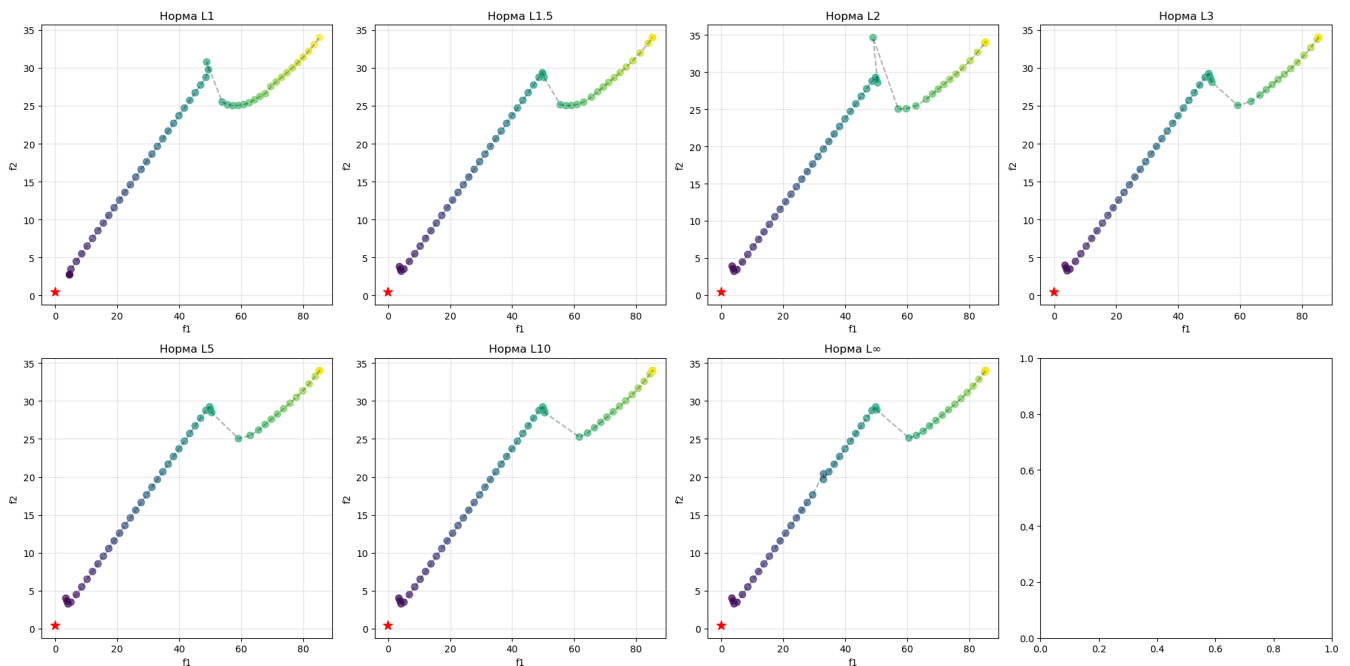
Норма L_∞ :

Количество решений: 50

диапазон f_1 : [3.479, 85.225]

диапазон f_2 : [3.240, 34.002]

Среднее расстояние: 4.129



Описание графиков:

Расширенная визуализация включает:

- Сравнение различных норм в пространстве критериев
- Решения в пространстве переменных
- Зависимость расстояния от положения целевой точки
- 3D визуализация критериев и нормы

- Линии уровня для разных норм
- Анализ чувствительности к выбору нормы

Анализ:

Различные нормы порождают существенно разные фронты Парето. L1-норма (манхэттенское расстояние) дает угловые решения, L2-норма (евклидово расстояние) - сглаженные траектории, а L^∞ -норма (чебышевское расстояние) стремится к равномерному приближению ко всем критериям. График линий уровня показывает геометрическую интерпретацию норм: L1 создает ромбовидные, L2 - круглые, L^∞ - квадратные линии уровня. Анализ чувствительности демонстрирует, что выбор нормы значительно влияет на разброс получаемых решений.

7. Сравнение методов

Сравнительный анализ всех методов:

```
# Генерируем данные для сравнения
methods_results = {
    'Взвешивания': objective_values,
    'ε-ограничений': np.column_stack([f1_vals, f2_vals]),
    'Идеальной точки': np.array([[f1(sol), f2(sol)] for sol in solutions_norm])
}

# Визуализация сравнения
plt.figure(figsize=(12, 8))

colors = {'Взвешивания': 'blue', 'ε-ограничений': 'green', 'Идеальной точки': 'red'}
markers = {'Взвешивания': 'o', 'ε-ограничений': 's', 'Идеальной точки': '^'}

for method, results in methods_results.items():
    plt.scatter(results[:, 0], results[:, 1],
                c=colors[method], marker=markers[method],
                s=80, label=method, alpha=0.7)

# Идеальная точка
plt.scatter(ideal[0], ideal[1], color='black', s=200, marker='*',
            label='Идеальная точка', zorder=10)

plt.xlabel('критерий f1')
plt.ylabel('критерий f2')
plt.title('Сравнение методов многокритериальной оптимизации')
plt.legend()
plt.grid(True, alpha=0.3)

# Добавляем аннотации
plt.annotate('лучшие по f1', xy=(ideal[0], ideal[1]),
            xytext=(ideal[0]+1, ideal[1]),
            arrowprops=dict(arrowstyle='->', color='gray'))
plt.annotate('лучшие по f2', xy=(ideal[0], ideal[1]),
            xytext=(ideal[0], ideal[1]+1),
```

```
arrowprops=dict(arrowstyle='->', color='gray'))
```

```
plt.tight_layout()
plt.show()
```

```
# Сводная таблица результатов
```

```
print("Сравнительная характеристика методов:")
```

```
print("Метод          | количество решений | диапазон f1   | диапазон f2")
```

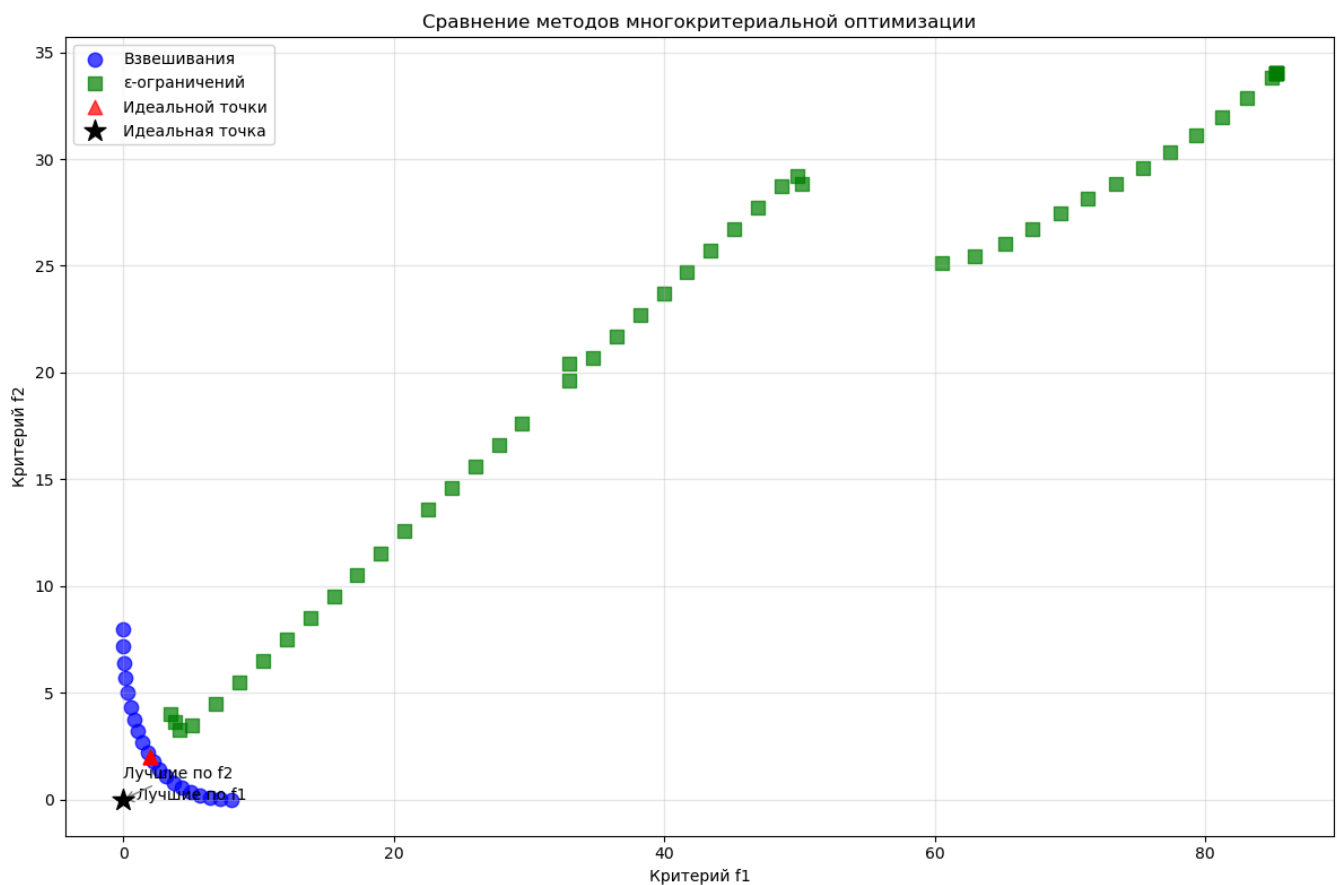
```
print("-" * 65)
```

```
for method, results in methods_results.items():
```

```
    f1_range = f"{results[:, 0].min():.2f} - {results[:, 0].max():.2f}"
```

```
    f2_range = f"{results[:, 1].min():.2f} - {results[:, 1].max():.2f}"
```

```
    print(f"{method:15} | {len(results):17} | {f1_range:12} | {f2_range:12}")
```



Сравнительная характеристика методов:

Метод	количество решений	диапазон f1	диапазон f2
-------	--------------------	-------------	-------------

Взвешивания	20	0.00 - 8.00	0.00 - 8.00
-------------	----	-------------	-------------

ε-ограничений	50	3.48 - 85.22	3.24 - 34.00
---------------	----	--------------	--------------

Идеальной точки	3	2.00 - 2.00	2.00 - 2.00
-----------------	---	-------------	-------------

Общие выводы по методам:

1. **Метод взвешивания** наиболее прост в реализации, но неэффективен для невыпуклых задач.
2. **Метод ε-ограничений** более универсален, но требует тщательного выбора параметров.

3. **Метод идеальной точки** интуитивно понятен, но сильно зависит от выбора нормы.
4. **Визуализация** играет ключевую роль в понимании компромиссов и выборе окончательного решения.
5. Комбинированное использование методов позволяет получить наиболее полное представление о множестве Парето.